CAPACITAÇÃO EM IOT E ANDROID EMBARCADO

Android Embarcado e Framework

Tópico 3: Kernel, HAL e Serviços Nativos

Prof. Isaías Felzmann - IC/Unicamp

Aluno(a)s: [adicione-nomes-aqui]

Nota:

Prática 3: Implementação e Teste de uma HAL para Acelerômetro

Objetivo: Desenvolver e integrar uma Hardware Abstraction Layer (HAL) em C/C++ para o sensor de acelerômetro.

Etapa 1: Conexão e Configuração do Ambiente

Execute os comandos abaixo no seu computador local para acessar o servidor de compilação do AOSP:

1. Conecte-se ao servidor de desenvolvimento:

```shell
ssh $USER -p 2222  froyo@androidbuilder.ic.unicamp.br
```

2. Inicie uma sessão persistente (evita perder o processo se a conexão cair):

```shell
byobu
```

3. Navega até o diretório do ambiente de desenvolvimento:

```shell
cd /data/aosp/android_course_environment
```

4. Carrega variáveis e funções necessárias para compilação:

```shell
source build/new_envsetup.sh
```

5. Selecione o alvo de compilação:

```shell
lunch aosp_rpi5-bp4a-userdebug
```

Etapa 2: Desenvolvimento da HAL

Você irá criar uma biblioteca compartilhada para a HAL e um programa cliente (CLI) para testá-la.

1. Crie o diretório do seu grupo:

```shell
cd hardware
mkdir acelerometro_<nome_do_grupo>
cd acelerometro_<nome_do_grupo>
```

2. Crie os arquivos base:

```shell
touch AcelerometroHAL.h AcelerometroHAL.cpp Android.bp acelerometro_cli.cpp
```

3. Implemente a interface da HAL (`AcelerometroHAL.h`):

```c/c++
#pragma once
#include <stdint.h>

#define ADDR_ACCEL 0x6A // Endereço I2C do LSM9DS1 (Accel/Gyro)

struct IMUData {
    float x;
    float y;
    float z;
};
class AcelerometroHAL {
public:
    AcelerometroHAL();
    ~AcelerometroHAL();
    bool init();
    IMUData readAccel();
private:
    int i2c_fd;
    bool openI2C();
    bool configureSensors();
    bool writeByte(uint8_t address, uint8_t reg, uint8_t value);
    bool readBlock(uint8_t address, uint8_t reg, uint8_t* buffer, int length);
};
```

4. Implemente a Lógica da HAL (`AcelerometroHAL.cpp`):

```C/C++
#include "AcelerometroHAL.h"
#include <cmath>
#include <fcntl.h>
#include <iostream>
#include <sys/ioctl.h>
#include <unistd.h>
#include <linux/i2c-dev.h>

AcelerometroHAL::AcelerometroHAL() : i2c_fd(-1) {}
AcelerometroHAL::~AcelerometroHAL() {
    if (i2c_fd >= 0)
        close(i2c_fd);
}
bool AcelerometroHAL::openI2C() {
  if (i2c_fd >= 0)
    return true;
  i2c_fd = open("/dev/i2c-1", O_RDWR);
  if (i2c_fd < 0) {
    std::cerr << "Failed to open /dev/i2c-1" << std::endl;
    return false;
  }
  return true;
}
bool AcelerometroHAL::init() {
  if (!openI2C())
    return false;
  return configureSensors();
}
bool AcelerometroHAL::configureSensors() {
  writeByte(ADDR_ACCEL, 0x10, 0x68);
  writeByte(ADDR_ACCEL, 0x20, 0x60);
  return true;
}
bool AcelerometroHAL::writeByte(uint8_t address, uint8_t reg, uint8_t value) {
  if (ioctl(i2c_fd, I2C_SLAVE, address) < 0)
    return false;
  uint8_t buf[2] = {reg, value};
  return write(i2c_fd, buf, 2) == 2;
}
bool AcelerometroHAL::readBlock(uint8_t address, uint8_t reg, uint8_t *buffer, int length) {
  if (ioctl(i2c_fd, I2C_SLAVE, address) < 0)
```

```
    return false;
  // For many sensors, setting MSB of register enables auto-increment
  uint8_t reg_cmd = reg | 0x80;
  write(i2c_fd, &reg_cmd, 1);
  return read(i2c_fd, buffer, length) == length;
}
IMUData AcelerometroHAL::readAccel() {
  uint8_t buf[6];
  if (!readBlock(ADDR_ACCEL, 0x28, buf, 6)) {
      std::cerr << "Failed to read accelerometer\n";
      return {0, 0, 0};
  }
  int16_t x = (int16_t)(buf[0] | (buf[1] << 8));
  int16_t y = (int16_t)(buf[2] | (buf[3] << 8));
  int16_t z = (int16_t)(buf[4] | (buf[5] << 8));
  return {x * 0.061f / 1000.0f, y * 0.061f / 1000.0f, z * 0.061f / 1000.0f};
}
```

5. Crie o Cliente de Teste (`acelerometro_cli.cpp`):

```
C/C++
#include "AcelerometroHAL.h"
#include <iostream>
#include <string>
void printUsage(const char* progName) {
    std::cout << "Usage: " << progName << " <command>\n"
              << "Commands:\n"
              << "  init    Unbind kernel driver and init sensor\n"
              << "  accelx  Read Accelerometer x-axis\n"
              << "  accely  Read Accelerometer y-axis\n"
              << "  accelz  Read Accelerometer z-axis\n";
}
int main(int argc, char* argv[]) {
    if (argc < 2) {
        printUsage(argv[0]);
        return 1;
    }
    std::string cmd = argv[1];
    if (cmd == "init") {
        // Desvincula o driver do kernel para permitir acesso em userspace
             int ret = system("echo '1-0046' > /sys/bus/i2c/drivers/simple-mfd-i2c/unbind 2>
/dev/null");
        (void)ret;
```

```
        std::cout << "Acelerometro initialized successfully (kernel driver unbound).\n";
        return 0;
    }
    AcelerometroHAL accel;
    if (!accel.init()) {
        std::cerr << "Error: Failed to initialize Acelerometro HAL.\n";
        return 1;
    }
    IMUData data = accel.readAccel();
    if (cmd == "accelx") {
        printf("Accelerometer X: %.3f\n", data.x);
    } else if (cmd == "accely") {
        printf("Accelerometer Y: %.3f\n", data.y);
    } else if (cmd == "accelz") {
        printf("Accelerometer Z: %.3f\n", data.z);
    } else {
        std::cerr << "Unknown command: " << cmd << "\n\n";
        printUsage(argv[0]);
        return 1;
    }
    return 0;
}
```

6. Configure o Build System (Android.bp):

Atenção: *substitua <nome_do_grupo> pelo mesmo nome usado nos passos anteriores*

```Shell
cc_library_shared {
    name: "libacelerometro_hal_<nome_do_grupo>",
    srcs: ["AcelerometroHAL.cpp"],
    vendor: true,
    cflags: ["-Wall", "-Werror"],
    export_include_dirs: [".“],
}
cc_binary {
    name: "acelerometro_<nome_do_grupo>",
    srcs: ["acelerometro_cli.cpp"],
    vendor: true,
    shared_libs: ["libacelerometro_hal_<nome_do_grupo>"],
    cflags: ["-Wall", "-Werror"],
}
```

Parte 3: Compilação e Testes

1. Compile o módulo: No servidor de desenvolvimento, retorne à raiz (`/data/aosp/android_course_environment`) e compile usando o comando m seguido do nome definido no `cc_binary`:

```shell
croot
```

Comando para compilação:

```shell
m acelerometro_<nome_do_grupo>
```

2. **Copie os binários compilados:** Salve os arquivos gerados para sua máquina local:
   - Biblioteca: `/data/aosp/android_course_environment/out_$USER/target/product/rpi5/vendor/lib64/libacelerometro_hal_<nome_do_grupo>.so`
   - Executável: `/data/aosp/android_course_environment/out_$USER/target/product/rpi5/vendor/bin/acelerometro_<nome_do_grupo>`
3. Conecte-se a placa via ADB:

```shell
adb pair <IP_DA_PLACA>:<PORTA>
adb root
adb remount
```

4. Dentro do que você salvou os binários na sua máquina local, envie os arquivos para a placa:

```shell
adb push libacelerometro_hal_<nome_do_grupo>.so /vendor/lib64/
```

<INSIRA PRINT>

```shell
adb push acelerometro_<nome_do_grupo> /vendor/bin/
```

<INSIRA PRINT>

5. Execute os testes na placa:

```shell
adb shell
```

Teste cada eixo:

```shell
acelerometro_<nome_do_grupo> accelx
acelerometro_<nome_do_grupo> accely
acelerometro_<nome_do_grupo> accelz
```

<INSIRA PRINT DO RESULTADO>

Relatório de Entrega

Seu relatório final deve conter a explicação dos seguinte itens:

- ☐ **Visão Geral da Arquitetura:** Descrição das camadas abordadas na prática
  - ☐ Hardware
  - ☐ Kernel
  - ☐ HAL
  - ☐ CLI
- ☐ **Descrição da Interface da HAL:** Explicação das funções implementadas (`openI2C`, `readBlock`, `readAccel`, etc.)
- ☐ **Dificuldades Encontradas:** Relato sobre eventuais problemas de compilação, comunicação I2C ou permissões no Android.